

Take-Home Assignment

“Chat with a Website” — Crawl + RAG

Full-Stack Developer

Time budget: ~8–10 hours of work, to be completed within 3–4 days of receiving this. Please don't over-invest — we would rather see a smaller, well-reasoned solution than an exhausted one.

AI tools: Fully allowed and encouraged. Use Cursor, Claude, Copilot, whatever you normally work with. We care about how you think and the decisions you make — not how fast you type. You will walk us through your code afterwards, so make sure you understand it.

Context

We build AI-powered tools that ingest external content and make it useful. This assignment shows us how you build a real retrieval pipeline around a language model — the parts the model can't do for you: crawling, cleaning, chunking, retrieval quality, and keeping answers honest.

The Task

Build an app where a user provides a **website URL**. The app crawls the site, indexes its **content, and then lets the user ask questions and get answers grounded in that site's content, with links back to the source pages.**

Keep the surface small — one site at a time and a simple chat box is plenty. The interesting work is the pipeline behind it, not the UI.

Core Requirements

- Accept a URL, crawl the site, and build a searchable index of its content.
- **Stay in scope:** crawl only within the given site/domain, with a sensible page or depth limit — don't wander off onto the wider internet.
- **Be polite:** respect robots.txt and rate-limit your requests. Don't hammer the target site.
- A chat interface where the user asks questions and gets answers based on the crawled content.
- Every answer cites its sources — link to the page(s) the answer came from.
- **Stay grounded:** when the site doesn't cover something, the app should say so rather than inventing an answer.

Note on AI model APIs — If any part of your solution needs an AI model API, you may use your own free or paid/Pro API key or account from any provider you already have access to.

Stretch Goals

Only if you have time. Bonuses, not requirements.

- Stripping boilerplate (nav, footer, cookie banners) so the index isn't polluted.
- Handling JavaScript-rendered pages.
- Streaming responses for a responsive feel.
- A basic eval: a handful of question/expected-source pairs to measure retrieval quality.

Tech Stack

Layer	Expectation
Frontend	React/Nextjs
Backend	Node.js.
Vector / search store	Your choice — a vector DB, pgvector, or even a simple in-memory index if you justify it.

What We Are Evaluating

The engineering around the model: a scoped, polite crawl; sensible chunking and retrieval; and grounded, cited answers that don't hallucinate. The raw model call is the easy 5% — the other 95% is the test.

Be honest in your README. Tell us what works, what doesn't, and what you'd improve. "Retrieval is weak on long pages and here's how I'd fix it" scores higher than pretending it's perfect.

Deliverables

1. A GitHub repository (or zip) with the full source.

2. A README covering: how to run it (incl. required env vars), and a short section on your crawling strategy, chunking/retrieval approach, and how you keep answers grounded.
3. Optional bonus: a deployed link.
4. A 30–45 minute walkthrough call where you screen-share and walk us through your pipeline and decisions.

Ground Rules

- Commit as you go — we like seeing the history.
- If something's ambiguous, make a reasonable call, note it in the README, and move on.
- Don't gold-plate. A small site that answers accurately with citations beats a big crawler that hallucinates.

Good luck — we're looking forward to the walkthrough.

"Chat with a Website" — Assignment